

فاز دوم پروژه اول درس شبکه‌های کامپیوتری

بابک حسینی محتشم

۸۱۰۱۰۱۴۰۸

پریا پاسه‌ورز

۸۱۰۱۰۱۳۹۳

۱۴۰۴/۱۰

بخش اول

ابتدا ماشین مجازی P4 را اجرا می‌کنیم و در ترمینال دستور زیر را وارد می‌کنیم:

```
cd ~/p4-learning/examples/simple_int
```

محتوای دایرکتوری حاصل به شکل زیر است:

```
p4@p4:~/p4-tools/p4-learning/examples/simple_int$ ls
log          p4app.json  pcap        receive.py  s2-commands.txt  send.py
network.py   p4src       README.md   s1-commands.txt  s3-commands.txt  topology.json
```

شکل ۱. محتوای پوشه simple_int

همان طور که خواسته شده در این مکان p4run را اجرا می‌کنیم و خروجی مشابه تصویر زیر است:

```

Default interface: h1-eth0      10.0.1.1      00:00:0a:00:01:01
*****
*****
Network configuration for: h2
Default interface: h2-eth0      10.0.2.2      00:00:0a:00:02:02
*****
*****
Network configuration for: h3
Default interface: h3-eth0      10.0.3.3      00:00:0a:00:03:03
*****
*****
Network configuration for: h4
Default interface: h4-eth0      10.0.3.4      00:00:0a:00:03:04
*****
Starting mininet CLI...

=====
Welcome to the P4 Utils Mininet CLI!
=====
Your P4 program is installed into the BMV2 software switch
and your initial configuration is loaded. You can interact
with the network using the mininet CLI below.

To inspect or change the switch configuration, connect to
its CLI from your host operating system using this command:
  simple_switch_CLI --thrift-port <switch thrift port>

To view a switch log, run this command from your host OS:
  tail -f <log_dir>/<switchname>.log
By default log directory is "./log".

To view the switch output pcap, check the pcap files in <pcap_dir>:
  for example run:  sudo tcpdump -xxx -r s1-eth1.pcap
By default pcap directory is "./pcap".

*** Starting CLI:
mininet>

```

شکل ۲. اجرای محیط mininet

بدین ترتیب وارد محیط mininet شدیم. در این محیط دستور زیر را وارد می‌کنیم:

```
mininet> h2 python3 receive.py
```

بدین ترتیب در خروجی ترمینال پیام زیر نوشته می‌شود:

sniffing on h2-eth0

پیام بالا به معنی شنود کردن روی ارنترفیس h2-eth0 است. برای ارسال پیام در این اینترفیس نمی‌توان از ping استفاده کرد زیرا ping مبتنی بر ICMP است ولی اطلاعات INT تنها در بسته‌های UDP درج می‌شوند. پس برای ارسال بسته‌های INT از اسکریپت send.py استفاده می‌کنیم. به همین منظور دستور زیر را وارد می‌کنیم:

h1 python3 send.py 10.0.2.2 "INT test" 5

که در آن 10.0.2.2 آدرس میزبان مقصد یعنی h2 است و "INT test" یک پیام دلخواه است و 5 تعداد بسته‌های ارسالی را نشان می‌دهد. خروجی حاصل از دستور بالا به شکل زیر است:

```
mininet> h1 python3 send.py 10.0.2.2 "INT test" 5
#### Ethernet ####
dst      = ff:ff:ff:ff:ff:ff
src      = 00:00:0a:00:01:01
type     = IPv4
#### IP ####
version  = 4
ihl      = 6
tos      = 0x0
len      = 40
id       = 1
flags    =
frag     = 0
ttl      = 64
proto    = udp
chksum   = 0x43be
src      = 10.0.1.1
dst      = 10.0.2.2
\options \
|#### INT ####
|copy_flag = 0
|optclass  = control
|option    = 31
|length    = 4
|count     = 0
|\int_headers\
#### UDP ####
sport    = 4321
dport    = 1234
len      = 16
chksum   = 0x4dd0
#### Raw ####
load     = 'INT test'

Sent 1 packets.
```

شکل ۳. خروجی حاصل از اجرای send.py

حال برای آنکه بتوانیم بسته‌ها را دریافت کنیم و در فایل بنویسیم و در عین حال بسته‌هایی را ارسال کنیم، receive.py را به صورت پس‌زمینه اجرا می‌کنیم و محتوای خروجی آن را در فایل می‌نویسیم و سپس فایل send.py را اجرا می‌کنیم تا ۵ بسته را ارسال کند.

به همین منظور ابتدا دستور زیر را وارد می‌کنیم:

h2 python3 receive.py > recv.log &

بدین ترتیب با اجرای receive.py در پس‌زمینه و نوشته شدن محتوای آن در فایل recv.log،

می‌توانیم فایل send.py را اجرا کنیم:

h1 python3 send.py 10.0.2.2 "INT test" 5

حال می‌توانیم خروجی را با دستور زیر بررسی کنیم:

h2 less recv.log

پنج نمونه بسته مشابه شکل زیر در خروجی قابل مشاهده است.

```
###[ IP ]###
version    = 4
ihl        = 8
tos        = 0x0
len        = 48
id         = 1
flags      =
frag       = 0
ttl        = 62
proto      = udp
chksum     = 0x62ba
src        = 10.0.1.1
dst        = 10.0.2.2
\options   \
|###[ INT ]###
| copy_flag = 0
tos        = 0x0
| option    = 31
| length    = 12
| count     = 2
| \int_headers\
| |###[ SwitchTrace ]###
| | swid     = 2
| | qdepth   = 0
| | portid   = 1
| |###[ SwitchTrace ]###
| | swid     = 1
| | qdepth   = 0
| | portid   = 2
|
###[ UDP ]###
sport      = 4321
dport      = 1234
len        = 16
chksum     = 0x4dd0
###[ Raw ]###
load       = 'INT test'
```

شکل ۴. یکی از بسته‌های دریافت شده

در قسمت INT می‌توان دو سویچ مبدأ و مقصد را مشاهده کرد. در این قسمت تمام سویچ‌های مسیر اضافه می‌شوند. هر سویچ در مسیر، اطلاعات محلی خود مانند شناسه، عمق صف و پورت

خروجی را به این قسمت بسته اضافه می‌کند. بدین ترتیب اطلاعات وضعیت شبکه می‌تواند درون خود بسته‌ها حمل شوند و نیازی به سیستم مانیتورینگ جداگانه نیست و به کمک INT می‌توانیم وضعیت لحظه‌ای شبکه در هر hop را بررسی کنیم.

```
available nodes are:
h1 h2 h3 h4 s1 s2 s3
mininet> net
h1 h1-eth0:s1-eth1
h2 h2-eth0:s2-eth1
h3 h3-eth0:s3-eth1
h4 h4-eth0:s3-eth2
s1 lo: s1-eth1:h1-eth0 s1-eth2:s2-eth2 s1-eth3:s3-eth3
s2 lo: s2-eth1:h2-eth0 s2-eth2:s1-eth2
s3 lo: s3-eth1:h3-eth0 s3-eth2:h4-eth0 s3-eth3:s1-eth3
```

شکل ۵. گره‌ها و ساختار شبکه

با بررسی ساختار شبکه و بررسی قسمت INT از لاگ می‌توان متوجه شد که گره h1 پیام خود را به سویچ s1 فرستاد و سویچ هم پیام را به s2 فرستاد و در نهایت s2 به h2 تحویل داد. در روش In-Band Network Telemetry یا INT شبکه اطلاعات اندازه‌گیری گرفته شده را در خود بسته‌ها می‌نویسد به جای آنکه بسته‌های مانیتورینگ جداگانه‌ای ارسال کند. بدین ترتیب به صورت ریزدانه و به ازای هر بسته وضعیت شبکه بررسی می‌شود و ترافیک جدیدی به دلیل بسته‌های مانیتورینگ ایجاد نمی‌شود.

بخش دوم

برای بخش دوم در ابتدا فایل `simple_int.p4` را در پوشه `p4src` باز می‌کنیم. قصد داریم تغییراتی در این فایل اعمال کنیم تا در هر سویچ مجموعه `Queue Depth` بسته‌های عبوری و تعداد کل بسته‌های عبوری در رجیسترهایی ذخیره شوند. به این منظور در این فایل دو رجیستر خواسته شده را تعریف می‌کنیم.

```
//total queue depth
register<bit<32>>(1) total_queue_depth;

//number of processed packets
register<bit<32>>(1) total_packets;
```

شکل ۶. تعریف رجیسترهای مجموعه `Queue Depth` و تعداد بسته‌ها در `simple_int.p4`

سپس همان طور که خواسته شده در تابع `MyIngress` در تابع `apply` خط‌هایی را مشابه تصویر زیر اضافه می‌کنیم که به ازای هر بسته وارد شده، مقدار عمق صف آن را به رجیستر اضافه می‌کند و مقدار رجیستر تعداد بسته‌ها را یکی افزایش می‌دهد.

```
bit<32> qdepth_32 = (bit<32>) standard_metadata.enq_qdepth;

bit<32> current_qdepth;
bit<32> current_pkt_count;

total_queue_depth.read(current_qdepth, 0);
total_packets.read(current_pkt_count, 0);

total_queue_depth.write(0, current_qdepth + qdepth_32);
total_packets.write(0, current_pkt_count + 1);
```

شکل ۷. خط‌های آپدیت عمق صف و تعداد بسته‌ها

برای مشاهده مقادیر این دو رجیستر مشابه قبل کدهای ارسال و دریافت بسته را اجرا می‌کنیم و به ترمینال دیگری می‌رویم. سپس با اجرای دستور زیر به سویچ `s1` با پورت `۹۰۹۰` متصل می‌شویم:

`Simple_switch_CLI –thrift-port 9090`

حال با دستور `register_read` می‌توانیم مقدار دو رجیستر را بخوانیم.

```
RuntimeCmd: register_read total_queue_depth  
register index omitted, reading entire array  
total_queue_depth= 0  
RuntimeCmd: register_read total_packets  
register index omitted, reading entire array  
total_packets= 17
```

شکل ۸. مقادیر رجیسترهای سویچ s1

در ابتدا ۱۲ بسته در این سویچ ثبت شده بود و با اجرای دستور ارسال، ۵ بسته دیگر اضافه شد و بدین ترتیب از درستی کارکرد تغییرات اطمینان حاصل کردیم.

در مورد مجموع عمق صف‌ها، از آنجا که هر یک از بسته‌ها عمق صف خود را صفر نشان می‌دهند، مجموع هم به درستی برابر صفر است. دلیل صفر بودن عمق صف هر بسته می‌تواند زمان پردازش ناچیز و در نتیجه ایجاد نشدن صف باشد زیرا حتی با افزایش نرخ و تعداد ارسال بسته‌ها، باز عمق صف گزارش شده توسط هر صف برابر صفر بود.

در ادامه همان طور که خواسته شده به منظور چاپ بسته‌های دریافت شده در گیرنده به ازای هر hop و همچنین میانگین عمق صف در hopها تغییرات زیر را در فایل `receive.py` اعمال کردیم.

```

def handle_pkt(pkt):
    #print("got a packet")
    #pkt.show2()

    traces = pkt[IPOption_INT].int_headers
    total_qdepth = 0
    trace_count = len(traces)
    print("--- INT packet received ---")

    for i, trace in enumerate(traces):
        switch_id = trace.swid
        qdepth = trace.qdepth
        egress_port = trace.portid

        total_qdepth += qdepth

        print(f"Hop {i}: Switch ID = {switch_id}, Queue Depth = {qdepth}, ",
              f"Engress Port = {egress_port}")
        print('-' * 50)

    if trace_count > 0:
        avg_qdepth = total_qdepth / trace_count
    else:
        avg_qdepth = 0
    print(f"Average Queue Depth: {avg_qdepth:.2f}")
    sys.stdout.flush()

```

شکل ۹. تغییرات اعمال شده در receive.py برای چاپ اطلاعات بسته‌ها

مشخص است که به ازای هر بسته اطلاعات موجود در قسمت INT آن را چاپ می‌کنیم و در نهایت میانگین عمق صف را هم چاپ می‌کنیم.

```

mininet> h2 less recv.log
sniffing on h2-eth0
--- INT packet received ---
Hop 0: Switch ID = 2, Queue Depth = 0, Engress Port = 1
-----
Hop 1: Switch ID = 1, Queue Depth = 0, Engress Port = 2
-----
Average Queue Depth: 0.00
--- INT packet received ---
Hop 0: Switch ID = 2, Queue Depth = 0, Engress Port = 1
-----
Hop 1: Switch ID = 1, Queue Depth = 0, Engress Port = 2
-----
Average Queue Depth: 0.00

```

شکل ۱۰. چاپ اطلاعات بسته‌های دریافت شده

مشابه قبل عمق صف‌ها و در نتیجه میانگین عمق صف‌ها هم صفر است که می‌تواند به دلیل زمان پردازش ناچیز بسته‌ها باشد.

حال برای ارسال بسته‌ها از طریق TCP یک کپی از `send.py` ایجاد می‌کنیم و در آن به جای UDP از TCP استفاده می‌کنیم.

```
pkt = Ether(src=get_if_hwaddr(iface), dst="ff:ff:ff:ff:ff:ff") / IP(
    dst=addr, options = IPOption_INT(count=0,
    int_headers=[])) / TCP(
    dport=1234, sport=4321, flags = "S") / sys.argv[2]
```

شکل ۱۱. بسته TCP ایجاد شده در فرستنده

به طور مشابه فایل گیرنده را هم کپی می‌کنیم که در آن بسته‌های TCP شنود شوند.

```
sniff(filter="tcp and port 4321", iface = iface,
    prn = lambda x: handle_pkt(x))
```

شکل ۱۲. شنود بسته‌های TCP در گیرنده

حال مشابه قبل فرستنده و گیرنده را این بار روی TCP اجرا می‌کنیم.

```
mininet> h2 less recv.log
sniffing on h2-eth0
--- INT packet received ---
Hop 0: Switch ID = 2, Queue Depth = 0,  Engress Port = 1
-----
Hop 1: Switch ID = 1, Queue Depth = 0,  Engress Port = 2
-----
Average Queue Depth: 0.00
--- INT packet received ---
Hop 0: Switch ID = 2, Queue Depth = 0,  Engress Port = 1
-----
Hop 1: Switch ID = 1, Queue Depth = 0,  Engress Port = 2
-----
Average Queue Depth: 0.00
--- INT packet received ---
Hop 0: Switch ID = 2, Queue Depth = 0,  Engress Port = 1
-----
Hop 1: Switch ID = 1, Queue Depth = 0,  Engress Port = 2
-----
Average Queue Depth: 0.00
```

شکل ۱۳. پیام‌های حاصل از دریافت بسته‌های TCP

مشابه قبل عمق صف‌ها صفر است که می‌تواند به دلیل پردازش بسیار سریع بسته‌ها و ایجاد نشدن صف باشد.

جمع‌بندی

در این تمرین متوجه شدیم می‌توان با قرار دادن سرآیندهایی در قسمت Options بسته‌ها، مانیتورینگ شبکه را به صورت لحظه‌ای بدون ایجاد ترافیک اضافی بررسی کرد. همچنین زبان p4 را یاد گرفتیم و با ایجاد تغییراتی در فایل‌های مربوط به سویچ‌ها، دو رجیستر برای ذخیره تعداد بسته‌های عبوری و مجموع عمق صف آن‌ها اضافه کردیم. سپس تغییراتی در کد مربوط به میزبان گیرنده اعمال کردیم تا قسمت INT سرآیند را استخراج کند و اطلاعات آن را چاپ کند و میانگین عمق صف‌ها را هم حساب کند. پس از چاپ اطلاعات هنگام ارسال و دریافت بسته‌ها، کدهای مربوط به ارسال و دریافت بسته‌ها را تغییر دادیم تا با پروتکل TCP به جای UDP کار کنند. یکی از چالش‌های اصلی صفر بودن عمق صف‌ها بود که پس از تلاش برای افزایش عمق صف‌ها با افزایش نرخ و تعداد بسته‌های ارسال و ناموفق بودن، نتیجه گرفتیم احتمالاً به دلیل سرعت بسیار بالای پردازش بسته‌ها در شبیه‌سازی، صف تشکیل نمی‌شود.